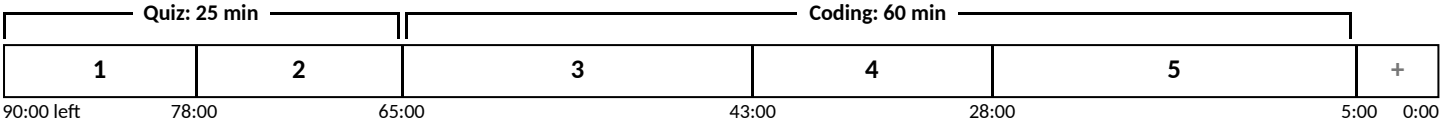
The test at a glance

CodeSignal data science assessment.
90 minutes, Python and quiz questions, no SQL.



Probability and statistics
2 quiz questions
12 min

ML fundamentals
4 to 6 quiz questions
13 min

Data collection
1 coding question: join files, save a table
22 min

Data processing
1 coding question: impute, encode, scale
15 min

Model development and evaluation
1 coding question: train, predict, beat a metric bar
23 min

Review
5 min

What the 90 minutes contain

Module	Qs	Format
1 Probability and statistics	2	Fill in the blank or multiple choice. Numeric answers rounded to 3 decimals.
2 Machine learning fundamentals	4-6	Multiple choice, often "select all that apply" from 5 or 6 options.
3 Data collection	1	Python. Combine several files into one table and save it in a stated format.
4 Data processing	1	Python. Impute, encode, scale, or bin, with no leakage from test into train.
5 Model development and evaluation	1	Python. Train a model and save predictions. The grader scores a metric such as MAE or precision on unseen data.

The three coding questions share one story (for example, houses and prices), but each starts from its own files. You can skip one and still do the next.

CodeSignal's framework lists 4 questions in module 2. Candidates report 5 or 6, and 10 or 11 questions in total.

How it is scored

- The score runs from 200 to 600.
- CodeSignal's scoring guide gives each module about the same base points, so two probability answers can match a whole coding question.
- Finishing every question in a module earns a bonus. Partial work still earns base points.
- Automated tests grade the coding questions. The model question is scored on data you can't see. Run your code early and often.
- There is no published pass mark. One candidate who estimated 60 to 70% correct passed.

Rules to expect

Confirm each one on your invite and on the rules screen before the timer starts.

- 90 minutes in one sitting. The link is usually valid for one week.
- Python only. pandas, NumPy, and scikit-learn are available.
- Proctored sessions record your camera, microphone, and screen, and ask for a government photo ID.
- One candidate's invite allowed official documentation and banned answer searches, AI tools, and other outside resources. Another reported that switching screens was not allowed. **Prepare for closed book.**
- This sheet is a revision aid. Use it during the test only if the on-screen rules allow notes.

Pacing

- Quiz first, capped at 25 minutes. These are the cheapest points per minute.
- If a quiz question passes 4 minutes, choose your best answer, note its number, and move on.
- Start coding with at least 65 minutes left.
- In each coding question, save a file in the right format before you improve anything.
- Keep at least 20 minutes for module 5. Candidates report running out of time there.

Ten traps that cost points

1. **Rounding.** Keep full precision to the end, then give exactly the decimals asked for.
2. **Over-selecting.** On "select all", judge each option alone as true or false.
3. **Output contract.** File name, column names, column order, dtypes, and no index column must match the question.
4. **Inner joins drop rows.** Left-join from the base table and compare row counts.
5. **Counts after a join.** No match gives NaN. Fill with 0, then cast to int.
6. **Date windows.** Both ends are usually inclusive, and timestamps carry a time of day.
7. **Leakage.** Fit imputers, encoders, and scalers on train only.
8. **Touching test rows.** Never drop, sort, or shuffle the test set. Predictions line up row for row.
9. **Polishing too early.** A plain model that runs beats a clever one that doesn't finish.
10. **Story first.** Read the required output first, then the story.

First two minutes of every coding question

1. Read the required output: file name, columns, and types.
2. List the files. For each one, look at `head()`, `dtypes`, and `isna().sum()`.
3. Write the skeleton that saves an output file with the right columns.
4. Run the tests, then fill in the logic.

Test-day checklist

Photo ID ..... on the desk
Laptop ..... plugged in, notifications off, other apps closed
Browser ..... current Chrome, one monitor
Camera and microphone ..... allowed and working
Network ..... strong Wi-Fi, phone hotspot ready
Room ..... quiet, door closed, 2 hours blocked
Paper, pen, calculator ..... only if the rules allow
Body ..... water ready, no breaks once the timer starts
Timing ..... rested, at least a day before the link expires
Warm-up ..... 15 minutes of pandas typing right before

Eight-day plan

Day	Do this
1	Practice kit, all 3 tasks, open book. List your gaps.
2	Page 6 template from memory, 3 times. Redo kit task 1.
3	Page 7 and 8 templates, 3 times. Redo kit tasks 2 and 3.
4	Mock 1: 90 minutes, closed book. Review every miss.
5	Fix your two weakest areas. 10 probability questions.
6	Pages 4 and 5 by recall. 20 select-all questions.
7	Mock 2 under test conditions. Check ID and laptop.
8	15-minute warm-up, then sit the test rested. Nothing new.

Every day: 15 minutes on pages 2 to 5. Cover the page and recite it.

Sources: CodeSignal Data Science Skills Evaluation Framework technical brief (2023); CodeSignal support articles on Assessment Score and proctoring; candidate write-ups from July 2024 and late 2025; Glassdoor, February 2026. Formats change, so trust your invite over this page.

Probability

Module 1 of 5. 2 questions, budget 12 of 90 minutes. Answers to 3 decimals.

Core rules

Complement	$P(A^c) = 1 - P(A)$
Addition	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Conditional	$P(A \mid B) = \dfrac{P(A \cap B)}{P(B)}$
Multiplication	$P(A \cap B) = P(A \mid B)\,P(B)$
Independence	$P(A \cap B) = P(A)\,P(B)$
Total probability	$P(B) = \sum_i P(B \mid A_i)\,P(A_i)$
Bayes	$P(A \mid B) = \dfrac{P(B \mid A)\,P(A)}{P(B)}$

- "At least one" = 1 - P(none).
- Mutually exclusive means $P(A \cap B) = 0$. That is not independence: two exclusive events with nonzero probability are dependent.
- Independent means that knowing B tells you nothing about A: $P(A \mid B) = P(A)$.

Trap. $P(A \mid B)$ is not $P(B \mid A)$. Write down which one the question asks for before you compute.

Bayes with counts

A condition affects 1% of people. The test catches 95% of cases and gives a false positive for 5% of healthy people. What is P(sick | positive)?

Take 10,000 people. 100 are sick and 95 of them test positive. 9,900 are healthy and 495 of them test positive.

$$P(\text{sick} \mid +) = \frac{95}{95 + 495} = \frac{0.95 \times 0.01}{0.95 \times 0.01 + 0.05 \times 0.99} = 0.161$$

A low base rate keeps the answer low even with an accurate test.

Counting

Choose k from n	Ways
Order matters, no repeats	$\dfrac{n!}{(n-k)!}$
Order doesn't matter, no repeats	$\binom{n}{k} = \dfrac{n!}{k!\,(n-k)!}$
Order matters, repeats allowed	n^k
Arrange n items with repeated groups	$\dfrac{n!}{n_1!\,n_2!\,\cdots}$

Which distribution

Situation	Model
n independent trials, same p, count the successes	Binomial
Draws **without replacement** from a finite pool	Hypergeometric
Trials until the first success	Geometric
Events in a fixed window at a known average rate	Poisson
Waiting time between such events (memoryless)	Exponential
Sum or mean of many independent values	Normal

Expectation and variance

- $E[X] = \sum_x x\,P(X = x)$. $\mathrm{Var}(X) = E[X^2] - (E[X])^2$.
- $E[aX + b] = a\,E[X] + b$. $\mathrm{Var}(aX + b) = a^2\,\mathrm{Var}(X)$.
- $E[X + Y] = E[X] + E[Y]$ always, independent or not.
- $\mathrm{Var}(X \pm Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) \pm 2\,\mathrm{Cov}(X, Y)$. Independent: the covariance is 0.
- $\mathrm{Cov}(X, Y) = E[XY] - E[X]\,E[Y]$. Correlation $= \mathrm{Cov}(X, Y) \,/\, (\sigma_X \sigma_Y)$, from -1 to 1.
- Total expectation: $E[X] = \sum_i E[X \mid A_i]\,P(A_i)$.
- **Indicator trick.** An expected count is the sum of each item's probability.
- Expected trials until the first success: $1 \,/\, p$. A six on a die takes 6 rolls on average.

Distribution facts

Distribution	P(X = k) or density	Mean	Variance
Bernoulli(p)	$p^k(1-p)^{1-k}$	p	$p(1-p)$
Binomial(n, p)	$\binom{n}{k} p^k (1-p)^{n-k}$	np	$np(1-p)$
Geometric(p)	$(1-p)^{k-1} p$	$\dfrac{1}{p}$	$\dfrac{1-p}{p^2}$
Hypergeometric (N, K, n)	$\dfrac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$	$\dfrac{nK}{N}$	$\dfrac{nK}{N}\left(1 - \dfrac{K}{N}\right)\dfrac{N-n}{N-1}$
Poisson(λ)	$\dfrac{e^{-\lambda}\lambda^k}{k!}$	λ	λ
Uniform(a, b)	$\dfrac{1}{b-a}$	$\dfrac{a+b}{2}$	$\dfrac{(b-a)^2}{12}$
Exponential(λ)	$\lambda e^{-\lambda x}$	$\dfrac{1}{\lambda}$	$\dfrac{1}{\lambda^2}$
Normal(μ, σ^2)	bell curve	μ	σ^2

Hypergeometric: N items in the pool, K successes, n drawn.

Normal distribution

- $z = (x - \mu) \,/\, \sigma$. 68% within 1 sd, 95% within 1.96 sd, 99.7% within 3 sd.
- P(Z < 1.645) = 0.95. P(Z < 1.96) = 0.975. P(Z < 2.576) = 0.995.
- Independent normals add: means add, and variances add (not standard deviations).
- Binomial is close to normal when np and $n(1-p)$ are both at least 10.

Worked examples

At least one. A six in 4 rolls: $1 - (5 \,/\, 6)^4 = 0.518$.

Binomial. Exactly 3 sixes in 10 rolls: $\binom{10}{3} (1 \,/\, 6)^3 (5 \,/\, 6)^7 = 0.155$.

With n = 30 and p = 0.7, P(X ≥ 25) = 0.077.

Without replacement. A 20-card deck holds 2 aces. Draw 5.

$$P(\text{no ace}) = \binom{18}{5} \,/\, \binom{20}{5} = \frac{15 \times 14}{20 \times 19} = 0.553$$

So P(at least one ace) = 0.447.

Multi-stage game (CodeSignal's published sample). Same deck. Draw 5. If you hold no ace, draw 3 new cards and shuffle the first 5 back. Then draw 1 more. P(at least one ace)?

Hit in the first 5: 0.447, done. Miss (0.553): the 3 new cards come from 15 that hold both aces and miss with $\binom{13}{3} \,/\, \binom{15}{3} = 0.629$. The last card comes from 17 and misses with 15/17 = 0.882.

$$0.447 + 0.553 \times (1 - 0.629 \times 0.882) = 0.693$$

Method: split on each stage, and track what is left in the deck.

Check answers in Python

```
from math import comb, factorial
from scipy.stats import binom, hypergeom, norm, poisson
comb(10, 3) * (1/6)**3 * (5/6)**7    # 0.155
binom.pmf(3, 10, 1/6)                # P(X = 3)
binom.sf(24, 30, 0.7)                # P(X >= 25) = 0.077
hypergeom.pmf(0, 20, 2, 5)           # pool, aces, draws
1 - norm.cdf(1.96); norm.ppf(0.975)  # 0.025 ; 1.96
poisson.pmf(0, 3)                    # e^-3 = 0.050
round(224 / 323, 3)                  # 0.693
```

Trap. Python's `round()` can turn a trailing 5 the wrong way: `round(2.675, 2)` gives 2.67. Check ties by hand.

Statistics and regression

Descriptive statistics

- Right-skewed data has mean > median. The median resists outliers; the mean doesn't.
- Sample variance divides by $n-1$; population variance divides by n. Standard deviation is its square root.
- IQR = Q3 - Q1. Outlier fences: Q1 - 1.5 IQR and Q3 + 1.5 IQR.
- z-score $= (x - \bar{x}) \,/\, s$. Coefficient of variation = sd / mean.
- Pearson r measures **linear** association only. r = 0 doesn't prove independence. Spearman works on ranks, so it catches any monotonic trend.
- Correlation is not causation. Look for a confounder.

Sampling and the central limit theorem

- The mean of n independent draws is close to normal for large n, whatever the population's shape, with

$$\mathrm{SE}(\bar{x}) = \frac{\sigma}{\sqrt{n}} \qquad \mathrm{SE}(\hat{p}) = \sqrt{\frac{p(1-p)}{n}}$$

- Four times the sample halves the standard error.
- Standard deviation describes the data. Standard error describes the estimate.

Confidence intervals

$$\text{estimate} \pm z^* \times \mathrm{SE}$$

- z^* = 1.645 for 90%, **1.96 for 95%**, 2.576 for 99%.
- Small n with unknown σ: use t with $n-1$ degrees of freedom. It is wider than z.
- Higher confidence widens the interval. Larger n narrows it.
- Meaning: 95% of intervals built this way contain the true value. It is not a 95% chance for this one interval.

> n = 100, mean 50, sd 10: $50 \pm 1.96 \times 10 \,/\, \sqrt{100}$ gives 48.04 to 51.96.
>
> 80 of 400 convert: $\hat{p} = 0.2$, SE = 0.02, so the 95% interval is 0.161 to 0.239.

Hypothesis tests

- p-value = P(data this extreme or more, given H0 is true). It is not P(H0 is true).
- Reject H0 when p < α. α = P(type I error) = false positive rate.
- β = P(type II error) = miss rate. Power = $1-\beta$.
- Power rises with a larger sample, a larger effect, a larger α, and less noise.
- Running m tests inflates false positives. Bonferroni tests each at $\alpha \,/\, m$.
- With a huge sample, a tiny effect is significant. Significant doesn't mean important.
- A two-sided p-value is twice the one-sided value when the test statistic is symmetric.
- A 95% confidence interval that excludes 0 matches p < 0.05 on the two-sided test.

Question	Test
One mean against a value, or two group means	t-test (z if n is large)
Before and after on the same subjects	Paired t-test
Three or more group means	ANOVA (F-test)
Two categorical variables, or fit to expected counts	Chi-square
Two proportions (A/B test)	Two-proportion z-test
Small, non-normal samples	Mann-Whitney, Wilcoxon

> **A/B test.** Control converts 200 of 2,000 (10%). The variant converts 250 of 2,000 (12.5%). The pooled rate is 450 / 4,000 = 0.1125.
>
> $$z = \frac{\hat{p}_2 - \hat{p}_1}{\sqrt{\hat{p}(1-\hat{p})\left(\frac{1}{n_1} + \frac{1}{n_2}\right)}} = \frac{0.025}{0.00999} = 2.50$$
>
> The two-sided p-value is 0.012, so you reject H0 at the 5% level.

Bias in data

- **Selection bias.** The sample doesn't represent the population. Example: asking only current customers why people leave.
- **Survivorship bias.** Only the survivors are observed, such as funds that still exist.
- **Non-response and self-selection.** The people who answer differ from the people who don't.
- **Confounding.** A third variable drives both X and Y. Random assignment removes it.
- **Simpson's paradox.** A trend inside every subgroup reverses when the groups are pooled, because group sizes differ.
- **Regression to the mean.** An extreme first measurement is usually followed by a less extreme one.

Linear regression

$$y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \varepsilon$$

- Ordinary least squares picks the coefficients that minimize the sum of squared errors.
- With one feature:

$$\hat{\beta}_1 = \frac{\mathrm{Cov}(x,y)}{\mathrm{Var}(x)} = r\frac{s_y}{s_x} \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \qquad R^2 = r^2$$

SSE $\sum_i (y_i - \hat{y}_i)^2$

MSE, RMSE $\dfrac{\mathrm{SSE}}{n}, \quad \sqrt{\mathrm{MSE}}$

MAE $\dfrac{1}{n}\sum_i |\, y_i - \hat{y}_i|$

R squared $1 - \dfrac{\mathrm{SSE}}{\mathrm{SST}}, \quad \mathrm{SST} = \sum_i (y_i - \bar{y})^2$

Adjusted $1 - (1-R^2)\dfrac{n-1}{n-p-1}$

> y = (3, 5, 7) and predictions (2, 5, 9). Errors are 1, 0, -2. SSE = 5, MSE = 1.667, RMSE = 1.291, MAE = 1, SST = 8, R squared = 0.375.

- **Coefficient.** A one-unit rise in x_j changes y by β_j, holding the other features fixed. With log y, it is about $100\,\beta_j$ percent.
- **Assumptions (LINE).** Linear relationship, Independent errors, Normal errors, Equal error variance. Also no perfect multicollinearity.
- **Multicollinearity** inflates coefficient variance and can flip signs. Predictions stay fine. Detect it with VIF above 5 to 10. Fix it by dropping or combining features, or with ridge.
- R squared never falls when you add a feature. Adjusted R squared can.
- Residual plot: a funnel means unequal variance; a curve means a missing non-linear term.
- Squared errors let outliers and high-leverage points pull the line.

Logistic regression

$$p = \frac{1}{1+e^{-z}}, \quad z = \beta_0 + \beta_1 x_1 + \cdots \qquad \ln\frac{p}{1-p} = z$$

- Odds $= p \,/\, (1-p)$. p = 0.8 gives odds of 4.
- A one-unit rise in x_j multiplies the odds by e^{β_j}. $\beta = 0.7$ doubles them (2.01).
- z = 0 gives p = 0.5. z = 2 gives 0.881. z = -2 gives 0.119.
- Fitted by maximum likelihood, which minimizes log loss. No closed form, and not least squares.
- The decision boundary is linear. The output is a probability; a label needs a threshold (0.5 by default).

$$\text{Log loss} = -\frac{1}{n}\sum_i \left[\, y_i \ln p_i + (1-y_i)\ln(1-p_i)\right]$$

- Always predicting 0.5 scores 0.693. A confident miss (p = 0.1 for a true 1) costs 2.303.

Machine learning fundamentals I

Diagnose the fit

What you see	Diagnosis	What helps
Training error high, validation error high	**Underfitting** (high bias)	Richer model, better features, less regularization, longer training
Training error low, validation error much higher	**Overfitting** (high variance)	More data, regularization, simpler model, early stopping, dropout, bagging
Training loss climbs every epoch	Learning rate too high	Lower it, scale the features, clip gradients
Training loss barely moves	Learning rate too low, or unscaled features	Raise it, scale, use momentum or Adam
Validation loss turns up while training loss still falls	Overfitting has started	Stop at the validation minimum
Validation score too good to be true	Leakage	Audit the features and the split
Good offline, poor in production	Leakage, skew, or drift	One pipeline for both, monitor, retrain

Bias-variance tradeoff. Expected error = bias squared + variance + irreducible noise. More model complexity lowers bias and raises variance. More data lowers variance, not bias.

Regularization

	L1 (lasso)	L2 (ridge)
Penalty	$\lambda \sum_j \lvert w_j \rvert$	$\lambda \sum_j w_j^2$
Effect	**Sets some weights exactly to 0.** Sparse model, built-in feature selection	Shrinks every weight toward 0, none exactly to 0
Correlated features	Keeps one, drops the others	Shares the weight among them
Solution	Iterative (not differentiable at 0)	Closed form
Prior (Bayesian view)	Laplace	Gaussian

- Elastic net mixes both penalties.
- Larger λ: simpler model, more bias, less variance. Too large underfits.
- scikit-learn: `alpha` is λ; `C` is $1 / \lambda$. A smaller `C` in LogisticRegression or SVC means stronger regularization.
- Standardize features first, or the penalty hits them unequally. The intercept isn't penalized.
- Other regularizers: early stopping, dropout, weight decay, limits on tree depth and leaf size, data augmentation, ensembling.

Gradient descent

$$w \leftarrow w - \eta \nabla L(w) \qquad \eta = \text{learning rate (step size)}$$

- **Batch** uses all rows per step: stable, slow. **Stochastic** uses one row: noisy, cheap. **Mini-batch** is the usual middle.
- η too large: the loss oscillates or diverges. Too small: slow, and it can stall.
- Momentum and Adam smooth and speed up the path. Scaled features make the loss surface rounder, so descent converges faster.
- Linear and logistic regression have convex losses with one global minimum. Neural networks don't.

Model families

Linear and logistic regression. A weighted sum of features. Fast and interpretable. Misses non-linear effects and interactions unless you add them as features. Scale when regularized.

k-nearest neighbors. Predicts from the k closest training points. No training step; prediction is slow and memory-heavy. Needs scaling. Degrades in high dimensions and with irrelevant features.

> **Choosing k.** Small k: flexible boundary, low bias, high variance, sensitive to noise. Large k: smoother, higher bias. k = n predicts the majority class. Choose k by cross-validation, use an odd k for two classes, and start near the square root of n.

Naive Bayes. Bayes' rule with features assumed independent given the class. Fast, and good with little data or text. Correlated features are double-counted, so probabilities are overconfident. An unseen feature value gives probability 0; Laplace smoothing fixes that.

> **Limits of Bayes' rule.** It needs a prior, which can be wrong or unknown. It needs P(evidence), so every alternative must be listed. The independence shortcut rarely holds.

Decision tree. Greedy splits that most reduce impurity: Gini or entropy for classes, variance for regression. No scaling needed. Captures non-linear effects and interactions, and is readable. High variance: a small data change gives a different tree. Control it with `max_depth`, `min_samples_leaf`, and pruning.

Random forest. Averages many deep trees, each grown on a bootstrap sample with a random subset of features per split. Out-of-bag rows give a free validation score.

Gradient boosting (GBM, XGBoost, LightGBM). Adds shallow trees in sequence, each fitted to the negative gradient of the loss.

Support vector machine. A maximum-margin boundary; kernels add non-linear shapes. Needs scaling. Small `C`: more regularization. Large `gamma`: overfits.

k-means. Unsupervised clustering around k centers. You choose k (elbow or silhouette). Needs scaling. Sensitive to starting points and outliers.

PCA. Unsupervised projection onto orthogonal directions of maximum variance. Scale first. It doesn't use the target.

Random forest vs gradient boosting

	Random forest (bagging)	Gradient boosting
Builds trees	Independently, in parallel	One at a time; each fits the errors of the ensemble so far
Trees	Deep	Shallow (weak learners)
Mainly reduces	**Variance**	**Bias**
More trees	Doesn't overfit; gains flatten	Can overfit; use a small learning rate and early stopping
Noisy labels, outliers	Robust	More sensitive
Tuning	Light	Learning rate, depth, tree count
Accuracy on tables	Strong baseline	Usually best when tuned

Ensembles. Bagging averages models trained on bootstrap samples. Boosting adds models in sequence. Stacking trains a meta-model on out-of-fold predictions. Voting takes the majority label (hard) or the mean probability (soft).

Feature preparation

- **Scaling matters** for distance and gradient methods: k-NN, k-means, SVM, PCA, regularized linear models, neural networks. **Trees don't need it.**
- Standardize: (x - mean) / sd. Min-max: to 0-1, outlier-sensitive. Robust: median and IQR.
- One-hot for unordered categories; drop one level for an unregularized linear model. Ordinal for ordered levels. Target encoding for many levels, fitted out of fold.
- Missing data: MCAR (pure chance), MAR (explained by observed columns), MNAR (depends on the missing value). Median for skewed numbers, mode for categories, plus a "was missing" flag.
- Curse of dimensionality: distances lose meaning and data needs explode. Select features or use PCA.
- Feature selection: filter (correlation, mutual information), wrapper (recursive elimination), embedded (L1, tree importance).

Machine learning fundamentals II

Classification metrics

	Predicted positive	Predicted negative
Actual positive	TP	FN (miss, type II)
Actual negative	FP (false alarm, type I)	TN

Precision $\frac{TP}{TP+FP}$ of predicted positives, the share that is right

Recall $\frac{TP}{TP+FN}$ of actual positives, the share you found

Specificity $\frac{TN}{TN+FP}$ and false positive rate = 1 - specificity

F1 $\frac{2PR}{P+R}$ harmonic mean; low if either one is low

Accuracy $\frac{TP+TN}{\text{all}}$ misleading when classes are imbalanced

> TP 40, FP 10, FN 20, TN 930: accuracy 0.970, precision 0.800, recall 0.667, F1 0.727, specificity 0.989.

- Recall is also called sensitivity and true positive rate.
- Costly false alarms: optimize precision. Costly misses: optimize recall.
- Raise the threshold: precision goes up, recall goes down.
- **ROC curve.** True positive rate against false positive rate over every threshold. **AUC** is the chance that a random positive scores above a random negative: 0.5 is random, 1 is perfect. It depends on ranking only, not on the threshold or calibration.
- With heavy imbalance, ROC AUC looks optimistic. Use the precision-recall curve (average precision).
- Gini = 2 × AUC - 1.
- **Log loss** scores the probabilities. It punishes confident mistakes and rewards calibration.

Regression metrics

	MSE and RMSE	MAE
Large errors	Squared, so they dominate	Counted in proportion
Outliers	Sensitive	Robust
Best constant guess	Mean	Median
Gradient	Smooth everywhere	Constant; undefined at 0

- RMSE and MAE are in the units of y. RMSE is never below MAE; a wide gap means a few large errors.
- R squared is the share of variance explained. It can be negative on new data.
- MAPE breaks when y is 0 and punishes over-forecasts more. Huber loss blends MSE and MAE.

Validation

- Train to fit, validation to choose, test to report once.
- **k-fold.** Split into k parts, train on k - 1, validate on the last, rotate, average. Steadier than one split on small data. 5 or 10 folds is standard.
- **Stratified** keeps class shares in every fold. **Group** keeps one entity's rows together. **Time series**: train on the past, validate on the future, never shuffle.
- **Leave-one-out** (k = n) has less bias but more variance and cost than 5-fold.
- **Tuning.** Grid search tries every combination. Random search covers a large space for less compute. Score both by cross-validation, never on the test set.
- Nested cross-validation gives an honest score when you also tune.

Data leakage

- A scaler, imputer, encoder, feature selector, or PCA fitted on all rows before the split.
- Target leakage: a feature that results from the target or is recorded after it.
- Time leakage: future rows in training, or a random split of time-ordered data.
- The same entity or duplicate rows on both sides of the split.
- Oversampling or SMOTE before the split. Target encoding computed on the rows it then predicts.
- Fix: split first, fit on train only, and put the steps in a pipeline inside cross-validation.

Class imbalance

- Stratify the split. Report precision, recall, F1, or PR AUC, not accuracy.
- Class weights make minority errors cost more. Resampling (oversample, SMOTE, undersample) applies to the training set only.
- Moving the threshold trades precision for recall with no retraining.
- Weights and resampling distort predicted probabilities.

Neural network fundamentals

- A layer computes activation(Wx + b). Without non-linear activations, any depth collapses to one linear model.
- **Activations.** Sigmoid: 0 to 1, saturates, gradients vanish. Tanh: -1 to 1, zero-centered, still saturates. ReLU: max(0, x), cheap, eases vanishing gradients, but units can die at 0. Leaky ReLU fixes dead units. Softmax: class probabilities that sum to 1.
- **Output and loss.** Regression: linear + MSE. Binary: sigmoid + binary cross-entropy. Multi-class: softmax + cross-entropy.
- Backpropagation applies the chain rule to get gradients. An optimizer (SGD, momentum, Adam) updates the weights.
- Vanishing or exploding gradients: ReLU, He or Xavier initialization, batch normalization, residual connections, gradient clipping.
- Regularize with dropout (training only), weight decay, early stopping, and data augmentation.
- All-zero starting weights make every unit learn the same thing.
- Epoch: one pass over the data. Small batches add noise that can help generalization; large batches are stable but need memory.

Answering "select all that apply"

- Judge each option alone, as its own true-or-false question.
- Absolute words (always, never, guarantees) are often false.
- A true statement that doesn't answer the question stays unselected.

True or false: statements that show up as options

- **T** L1 can set weights exactly to zero.
- **T** Bagging mainly cuts variance; boosting mainly cuts bias.
- **T** More trees can overfit gradient boosting.
- **T** AUC doesn't change if you rescale scores but keep their order.
- **T** Fitting a scaler before the split leaks.
- **T** MSE reacts to outliers more than MAE.
- **T** Lowering the threshold raises recall.
- **T** Standardizing changes k-NN and SVM results.
- **F** More trees overfit a random forest.
- **F** Scaling changes a decision tree's splits.
- **F** A larger k in k-NN raises variance. (It raises bias.)
- **F** A p-value is the probability that H0 is true.
- **F** Training R squared can drop when you add a feature.
- **F** A higher C means stronger regularization.
- **F** PCA uses the target.
- **F** Logistic regression is fitted by least squares.
- **F** Dropout stays on at prediction time.
- **F** k-means always finds the global optimum.
- **F** Multicollinearity biases OLS predictions.
- **F** Accuracy is a safe metric at 1% positives.
- **F** ROC AUC depends on the threshold you pick.
- **F** Early stopping increases overfitting.

Data collection with pandas

The pattern

Filter. Aggregate to one row per key. Left-join onto the base table. Fix nulls, names, types, and column order. Save without the index.

Below: CodeSignal's published sample. One row per house with its views in the last 30 days.

```
import pandas as pd
houses = pd.read_csv('houses.csv')
conds  = pd.read_csv('house_conditions.csv')
prices = pd.read_csv('house_prices.csv')
views  = pd.read_csv('house_views.csv',
                     parse_dates=['view_date'])

# 1 Filter: 16 Mar to 15 Apr 2023, both days included
d = views['view_date']
recent = views[(d >= '2023-03-16') & (d < '2023-04-16')]

# 2 Aggregate to one row per key
n_views = (recent.groupby('house_id').size()
                 .reset_index(name='num_views'))

# 3 Left-join everything onto the base table
out = (houses
       .merge(n_views, on='house_id', how='left')
       .merge(conds, on='condition_id', how='left')
       .merge(prices, on='house_id', how='left'))
assert len(out) == len(houses)   # no rows lost or doubled

# 4 Fix nulls, names, types, and column order
out['num_views'] = out['num_views'].fillna(0).astype(int)
out = out.rename(columns={'condition': 'condition_name'})
cols = ['house_id', 'bedrooms', 'bathrooms', 'sqft_living',
        'sqft_lot', 'floors', 'waterfront', 'age',
        'num_views', 'condition_name', 'price']
out[cols].to_csv('collected.csv', index=False)
```

Traps

- `merge` defaults to `how='inner'`, which drops unmatched rows.
- Duplicate keys on the right side multiply rows. Compare `len()` before and after.
- `groupby` drops rows whose key is NaN unless you pass `dropna=False`.
- One NaN turns an int column into float, so 3 saves as 3.0.
- `<= '2023-04-15'` on a timestamp cuts that day off after midnight.
- `df[mask]['x'] = 1` changes nothing. Use `df.loc[mask, 'x'] = 1`.
- `df['x'].fillna(0, inplace=True)` can change nothing. Assign the result back.
- Join keys need the same dtype. Strip spaces and fix case before you join or group.

pandas by task

Load and look

```
import os; print(os.listdir('.'))       # what did I get?
print(open('orders.csv').read(300))     # peek: sep, header
df = pd.read_csv('orders.csv', parse_dates=['order_date'])
cust = pd.read_json('cust.json')  # read_excel, read_parquet
# read_csv options: sep=';'  dtype={'id': str}  usecols=[..]
#   na_values=['?', 'NA']  encoding='latin-1'  skiprows=1
# JSON lines: pd.read_json(f, lines=True)
# Nested JSON: pd.json_normalize(json.load(open(f)))
df.shape; df.dtypes; df.head(3); df.isna().sum()
df.nunique(); df.duplicated().sum(); df.describe()
df['status'].value_counts(dropna=False)
```

Filter rows (WHERE)

```
df[(df.amount > 100) & (df.status == 'paid')]   # & | ~
df[df.city.isin(['SG', 'KL'])]          # ~ in front negates
df[df.amount.between(10, 99)]           # both ends included
df[df.city.str.contains('new', case=False, na=False)]
df.query("amount > 100 and status == 'paid'")
df[df.city.notna()]
df.loc[df.amount < 0, 'amount'] = 0     # conditional update
```

Dates

```
d = pd.to_datetime(df['order_date'], errors='coerce')
df[(d >= '2023-03-16') & (d < '2023-04-16')]  # to 15 Apr
d.dt.year; d.dt.month; d.dt.quarter
d.dt.dayofweek                          # Monday = 0
d.dt.to_period('M')                     # month bucket
(df.ship_date - df.order_date).dt.days
cutoff = d.max() - pd.Timedelta(days=30)   # last 30 days
```

Aggregate (GROUP BY)

```
g = df.groupby('cust_id', as_index=False).agg(
        n_orders=('order_id', 'count'),
        total=('amount', 'sum'),
        avg=('amount', 'mean'),
        n_cities=('city', 'nunique'),
        last=('order_date', 'max'))
g[g.n_orders >= 3]                      # HAVING
(df.groupby(['city', 'status']).size()
   .reset_index(name='n'))              # COUNT(*)
# count = non-null values, size = rows, nunique = distinct
```

Window functions

```
df = df.sort_values(['cust_id', 'order_date'])
by = df.groupby('cust_id')['amount']
df['cust_total'] = by.transform('sum')      # SUM() OVER
df['rnk'] = by.rank(method='dense', ascending=False)
df['prev'] = by.shift(1)                    # LAG
df['running'] = by.cumsum()
df['seq'] = df.groupby('cust_id').cumcount() + 1
df['roll3'] = df['amount'].rolling(3).mean()
```

Top N and duplicates

```
df.nlargest(5, 'amount')
(df.sort_values('amount', ascending=False)
   .groupby('city').head(3))            # top 3 per group
(df.sort_values('order_date')
   .drop_duplicates('cust_id', keep='last'))   # latest row
df.drop_duplicates(subset=['order_id'])
```

Join

```
m = df.merge(cust, on='cust_id', how='left')
#   how = inner (default) | left | right | outer
m = df.merge(people, left_on='cust_id', right_on='id',
             how='left', suffixes=('', '_p'))
df.merge(cust, on='cust_id', how='left', validate='m:1')
#   validate raises if the right side has duplicate keys
a = df.merge(cust, on='cust_id', how='left',
             indicator=True)
a[a._merge == 'left_only']              # anti-join
pd.concat([df1, df2], ignore_index=True)    # UNION ALL
```

Clean and transform

```
df['city'] = df['city'].str.strip().str.lower()
df['amount'] = pd.to_numeric(df['amount'], errors='coerce')
df['qty'] = df['qty'].fillna(0).astype(int)  # after fillna
df = df.dropna(subset=['cust_id'])
df['tier'] = df['code'].map({'L': 1, 'M': 2, 'H': 3})
df['big'] = np.where(df.amount > 100, 1, 0)    # CASE WHEN
df['band'] = np.select(
    [df.amount < 10, df.amount < 100], ['low', 'mid'],
    default='high')
df['bin'] = pd.cut(df.amount, [0, 10, 100, np.inf],
                   labels=['low', 'mid', 'high'])
df = df.rename(columns={'amt': 'amount'})
df.columns = (df.columns.str.strip().str.lower()
                .str.replace(' ', '_'))
df['ref'].str.extract(r'(\d+)')[0].astype(float)
```

Reshape

```
wide = df.pivot_table(index='city', columns='status',
    values='amount', aggfunc='sum', fill_value=0)
long = wide.reset_index().melt(id_vars='city')
```

Shape and save

```
out = g[['cust_id', 'n_orders', 'total']]  # names, order
out = out.sort_values(['total', 'cust_id'],
                      ascending=[False, True])
out = out.astype({'cust_id': int}).round({'total': 2})
out.to_csv('result.csv', index=False)      # never the index
```

Data processing without leakage

The rule

> Learn every statistic on train. Apply it to train and test. The test set only ever gets `transform`.

The code below solves CodeSignal's published sample: fill missing ages with the mean, ordinal-encode the condition, and standard-scale two columns.

```
import pandas as pd
from sklearn.preprocessing import (OrdinalEncoder,
                                   StandardScaler)

train = pd.read_csv('data/train.csv')
test  = pd.read_csv('data/test.csv')

# 1 Impute: the statistic comes from train only
age_mean = train['age'].mean()
train['age'] = train['age'].fillna(age_mean)
test['age']  = test['age'].fillna(age_mean)

# 2 Ordinal encode: fit on train, transform both
cat = ['condition_name']
enc = OrdinalEncoder()      # alphabetical unless you say
train[cat] = enc.fit_transform(train[cat])
test[cat]  = enc.transform(test[cat])

# 3 Standard scale: fit on train, transform both
num = ['sqft_living', 'sqft_lot']
sc = StandardScaler()
train[num] = sc.fit_transform(train[num])
test[num]  = sc.transform(test[num])

# 4 Save both files
train.to_csv('processed_train.csv', index=False)
test.to_csv('processed_test.csv', index=False)
```

What the grader can check

- Same rows, same row order, and same columns as the input files.
- No missing values left in the imputed columns.
- Test rows filled and scaled with **train** statistics. Scaled train columns have mean 0 and sd 1; test columns don't.
- Encoded values that match the stated mapping, exact file names, and no index column.

Traps

- pandas `.std()` divides by n - 1. `StandardScaler` divides by n. Pass `ddof=0` to match.
- `OrdinalEncoder()` sorts levels alphabetically. If the question gives an order, pass it in `categories`.
- A level that appears only in test raises an error unless you set `handle_unknown`.
- `get_dummies` on train and test separately gives different columns. Reindex test to train's columns.
- Transform only the columns the question names. Leave IDs, the target, and 0/1 flags alone.
- Never drop rows from test, even outliers or rows with missing values.
- Follow the order of steps in the question: impute before you scale.
- A transformer returns a NumPy array. Assign it back to the same column list to keep the DataFrame.

NumPy and Python reminders

```
a = np.array([[1, 2, 3], [4, 5, 6]])
a.sum(axis=0)       # down the rows: one value per column
a.mean(axis=1)      # across the columns: one value per row
np.nanmean(x); np.isnan(x).sum()   # NaN == NaN is False
np.percentile(x, [25, 50, 75]); np.clip(x, lo, hi)
X = x.reshape(-1, 1)      # 2-D column for scikit-learn
vals, counts = np.unique(labels, return_counts=True)
rng = np.random.default_rng(42); rng.permutation(5)
sorted(rows, key=lambda r: (-r['n'], r['id']))  # desc, asc
{k: v for k, v in zip(vals, counts)}; [v * 2 for v in vals]
```

Techniques

Impute

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='median')
#   strategy = mean | median | most_frequent | constant
train[num] = imp.fit_transform(train[num])
test[num]  = imp.transform(test[num])
# pandas: number, category, and a missing flag
train['was_na'] = train['x'].isna().astype(int)
med  = train['x'].median()
mode = train['c'].mode()[0]
train['x'] = train['x'].fillna(med)
test['x']  = test['x'].fillna(med)
# group-wise: a per-group median learned on train
gm = train.groupby('g')['y2'].median()
test['y2'] = (test['y2'].fillna(test['g'].map(gm))
                        .fillna(train['y2'].median()))
```

Encode

```
from sklearn.preprocessing import (OneHotEncoder,
                                   OrdinalEncoder)
# One-hot. Older scikit-learn: sparse=False
oh = OneHotEncoder(handle_unknown='ignore',
                   sparse_output=False)
A = oh.fit_transform(train[cat])
B = oh.transform(test[cat])
names = oh.get_feature_names_out(cat)
train = train.drop(columns=cat).join(
    pd.DataFrame(A, columns=names, index=train.index))
test = test.drop(columns=cat).join(
    pd.DataFrame(B, columns=names, index=test.index))
# One-hot in pandas: align test to train's columns
tr = pd.get_dummies(train2, columns=cat)
te = (pd.get_dummies(test2, columns=cat)
        .reindex(columns=tr.columns, fill_value=0))
# Ordinal with a stated order, unseen levels to -1
oe = OrdinalEncoder(
    categories=[['low', 'mid', 'high']],
    handle_unknown='use_encoded_value', unknown_value=-1)
# or: df['lvl'].map({'low': 0, 'mid': 1, 'high': 2})
```

Scale and transform

```
from sklearn.preprocessing import (StandardScaler,
    MinMaxScaler, RobustScaler, PowerTransformer)
StandardScaler()    # (x - mean) / std, std uses ddof=0
MinMaxScaler()      # (x - min) / (max - min), range 0 to 1
RobustScaler()      # (x - median) / IQR, outlier-proof
PowerTransformer()  # Yeo-Johnson, makes x more normal
np.log1p(df['x'])   # right-skewed, x >= 0; undo: np.expm1

# same as StandardScaler, in pandas
mu, sd = train['x'].mean(), train['x'].std(ddof=0)
train['x'] = (train['x'] - mu) / sd
test['x']  = (test['x'] - mu) / sd
```

Bin (discretize)

```
# Fixed edges: [0, 18), [18, 40), [40, 65), [65, inf)
edges = [0, 18, 40, 65, np.inf]
df['age_bin'] = pd.cut(df['age'], bins=edges,
                       labels=False, right=False)
# Quantile bins: learn the edges on train, reuse on test
_, q = pd.qcut(train['x'], 4, retbins=True)
q[0], q[-1] = -np.inf, np.inf     # catch unseen extremes
train['x_bin'] = pd.cut(train['x'], bins=q, labels=False)
test['x_bin']  = pd.cut(test['x'], bins=q, labels=False)
```

Outliers

```
# IQR fences from train
q1, q3 = train['x'].quantile([0.25, 0.75])
iqr = q3 - q1
lo, hi = q1 - 1.5 * iqr, q3 + 1.5 * iqr
train['x'] = train['x'].clip(lo, hi)   # cap (winsorize)
test['x']  = test['x'].clip(lo, hi)
# or drop them, in train only:
train = train[train['x'].between(lo, hi)]
# z-score rule, also with train statistics
z = (df['x'] - train['x'].mean()) / train['x'].std()
df['is_out'] = z.abs() > 3
```

Model development and evaluation

Order of work

1. Read the metric, the target, and the output format.
2. Save a valid predictions file from a baseline within 5 minutes.
3. Compare linear, random forest, and boosting on validation. Beat a median-only baseline first.
4. Match the loss or the threshold to the metric.
5. Refit the winner on train plus validation.
6. Predict test, save, and check the file.

Regression template

Below: CodeSignal's published sample, scored on MAE.

```
import pandas as pd
from sklearn.linear_model import Ridge
from sklearn.ensemble import (RandomForestRegressor,
    HistGradientBoostingRegressor)
from sklearn.metrics import mean_absolute_error
# 1 Load, name the target, list the features
train = pd.read_csv('train.csv')
val   = pd.read_csv('val.csv')
test  = pd.read_csv('test.csv')
target = 'price'
feats = list(test.columns.drop('house_id'))

# 2 Compare three model families on validation
models = {
    'ridge': Ridge(),
    'rf': RandomForestRegressor(
        n_estimators=300, random_state=42, n_jobs=-1),
    'hgb': HistGradientBoostingRegressor(
        loss='absolute_error', random_state=42)}
scores = {}
for name, m in models.items():
    m.fit(train[feats], train[target])
    scores[name] = mean_absolute_error(
        val[target], m.predict(val[feats]))
    print(name, scores[name])

# 3 Refit the winner on train + val, then predict test
best = models[min(scores, key=scores.get)]
full = pd.concat([train, val], ignore_index=True)
best.fit(full[feats], full[target])
out = pd.DataFrame({'price': best.predict(test[feats])})
out.to_csv('predictions.csv', index=False)
```

HistGradientBoosting accepts NaN; Ridge and older random forests don't.

Match the lever to the metric

The question scores	Do this
MAE	`loss='absolute_error'`
RMSE, MSE, R squared	Default squared-error loss
Long right tail in y	Fit on `np.log1p(y)`, predict with `np.expm1`
Accuracy	Labels at the 0.5 threshold
A precision bar	Raise the threshold. No class weights
A recall bar	Lower the threshold, or weight the classes
F1	Scan thresholds on validation, keep the best
AUC or log loss	Save `predict_proba` scores, not labels

If the score is short of the bar

- Look for useful columns you dropped, and IDs or leaks you kept.
- Add obvious features (ratios, date parts), then tune depth, learning rate, and tree count on 3 folds. Six to twelve combinations is enough.

Before you submit

Rows ..... same count and order as the test file
Header ..... exact column name from the question
Index ..... not saved: `index=False`
Values ..... no NaN; labels or probabilities as asked
File ..... exact name and folder
Run ..... the whole script runs top to bottom

Classification, validation, metrics

The pipeline imputes, encodes, and fits in one object, so nothing leaks in cross-validation.

Pipeline for mixed columns

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier

X, y = train[feats], train[target]
num = X.select_dtypes('number').columns
cat = X.columns.difference(num)

pre = ColumnTransformer([
    ('num', SimpleImputer(strategy='median'), num),
    ('cat', make_pipeline(
        SimpleImputer(strategy='most_frequent'),
        OneHotEncoder(handle_unknown='ignore')), cat)])
clf = make_pipeline(pre, RandomForestClassifier(
    n_estimators=300, random_state=42, n_jobs=-1))
#   class_weight='balanced': more recall, less precision
clf.fit(X, y)
```

Hit a precision or recall bar

```
from sklearn.metrics import precision_score, recall_score
y_val = val[target]
p_val = clf.predict_proba(val[feats])[:, 1]
for t in [0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]:
    lab = (p_val >= t).astype(int)
    print(t, lab.sum(),
          precision_score(y_val, lab, zero_division=0),
          recall_score(y_val, lab))
t = 0.7      # lowest t that clears the bar with a margin
p_test = clf.predict_proba(test[feats])[:, 1]
pred = (p_test >= t).astype(int)
pd.DataFrame({target: pred}).to_csv(
    'predictions.csv', index=False)
```

Split, cross-validate, tune

```
from sklearn.model_selection import (train_test_split,
    cross_val_score, KFold, StratifiedKFold, GridSearchCV)
X_tr, X_va, y_tr, y_va = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
#   stratify=y for classification only
cv = StratifiedKFold(5, shuffle=True, random_state=42)
#   regression: KFold(5, shuffle=True, random_state=42)
#   leave-one-out: LeaveOneOut(), small data only
s = cross_val_score(clf, X, y, cv=cv, scoring='f1')
print(s.mean(), s.std())
#   scoring: accuracy precision recall f1 roc_auc r2
#   neg_log_loss neg_mean_absolute_error
grid = GridSearchCV(clf, cv=3, scoring='f1', n_jobs=-1,
    param_grid={
        'randomforestclassifier__max_depth': [6, 12, None],
        'randomforestclassifier__min_samples_leaf': [1, 5]})
grid.fit(X, y)
print(grid.best_params_, grid.best_score_)
best = grid.best_estimator_        # already refit on all X
```

Metrics in code

```
from sklearn.metrics import (mean_absolute_error,
    mean_squared_error, r2_score, accuracy_score,
    precision_score, recall_score, f1_score, roc_auc_score,
    log_loss, confusion_matrix, classification_report)
mae  = mean_absolute_error(y_true, y_pred)
rmse = mean_squared_error(y_true, y_pred) ** 0.5
r2   = r2_score(y_true, y_pred)
adj  = 1 - (1 - r2) * (n - 1) / (n - k - 1)   # k features
auc  = roc_auc_score(yc, proba)     # scores, not labels
gini = 2 * auc - 1
ll   = log_loss(yc, proba)
tn, fp, fn, tp = confusion_matrix(yc, label).ravel()
print(classification_report(yc, label))
```

Long right tail in the target

```
m = HistGradientBoostingRegressor(random_state=42)
m.fit(train[feats], np.log1p(train[target]))
pred = np.expm1(m.predict(val[feats]))   # back to y units
```